Politecnico di Torino

Politecnico di Torino

Corso di Laurea Ingegneria Informatica **LM-32 (DM270)**
A.a. 2023/2024

Mobile Application Developement

Answers to Past Exam Questions

Pierpaolo Bene (s319841)

Indice

2023-09-19

1.Android allows certain operations to be done in some threads, and some others not. How and why coroutines help in this sense? Give also an example.

All operations carried out in main thread must be short. In-fact long lasting operations can cause the displaying of “Application not responding” dialog. So in order to manage long tasks, programmers should create a new thread.
A possible approach is to use the Java library abstractions, wich provides locks, barrier, etc. Android also provides useful abstractions like **Looper** and **Handler** to support the programmers.
They are highly reliable but still very low level:

- Sharing the same address space can cause **interference** problems;
- **Asynchronous** communication needs polling or blocking approach to let a thread knows if some state has changed;
- Only the main thread can access to some part of Android framework, making other threads impossible to be aware of application **lifecycle**;
- Are hard to **cancel**, you need to poll the interrupted flag of the thread object which is hard to manage by the programmer and can lead to errors.

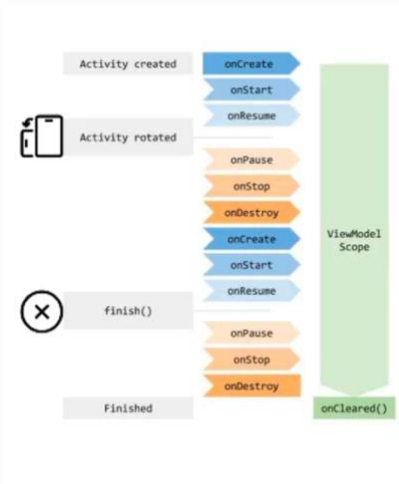
So the best alternative offered by Kotlin is to use **Coroutines**: functions that has one or more suspension points. Practically coroutines have a start, and at each suspension point the function will save the local state and give controls back to the caller, until it is resumed possibly in different thread from the one in which it started. With this mechanism coroutines provide an abstraction layer on top of system threading. The power of it is that after a coroutine is suspended, the thread can process different computation.
When coroutine cannot proceeds it may **suspend** itself , but before doing that it first arranges a way to resume. This enables possibility to express sequential logic, which is easier to write and understand for programmers, while dealing with asynchronous computation, which allows not to block on the main thread. In addition it solves the cancelability problems of system threads.

```
@Composable
fun MyScreen() {
    var isLoading by remember { mutableStateOf(false) }
    var data by remember { mutableStateOf("") }

    Column {
        if (isLoading) {
            // Show a progress indicator while loading data
            CircularProgressIndicator()
        } else {
            // Show the loaded data
            Text(data)
        }

        Button(onClick = {
            isLoading = true
            CoroutineScope(Dispatchers.Default).launch {
                // Simulate loading data
                delay(2000)

                // Update data on the main thread
                withContext(Dispatchers.Main) {
                    data = "Loaded data"
                    isLoading = false
                }
            }
        }) {
            Text("Load Data")
        }
    }
}
```

In this example the Composable MyScreen hold two states (isLoading, data). When the Button is pressed the view show a Circular Progress indicator, while a Coroutine fetch some data. When The Data is returned from the db, the isLoading move to false triggering a recomposition and showing the retrieved Text.

2. View Model, what is its role and how is its lifecycle compared to the other components it interacts with.

The **View Model** is a central component in the MVVM (Model – View – View Model) architecture pattern. Its main role is to store and manage UI-Related data in a lifecycle conscious way and to propagate them to the View, which holds the user interface.



It allows to rebind the same data to different activities/fragments when they are destroyed and recreated (like when you rotate the device).

Regarding the lifecycle , While activities has onCreate, onStart, onResume, onPause, onStop, onDestroy…. View Model instances have a special status in Android since they are created in association with a fragment or an activity and are retained since the scope is alive. So it is kept in memory and associated back to their scope as soon as they're recreated. When the tied activity/fragment is completely destroyed, the ViewModel is notified via the **onCleared()** method.

3. Jetpack Compose library

The standard approach is limited because it splits the tree view code among several files: layout resource, activities, fragments and other resources files. In addition **data flow is not specified** , how information is propagated through views is a programmer choice.

Jetpack Compose , instead, provides a reactive programming model based on **composable** functions. Compose is fully declarative and data driven, it works separating UI in single UI components (buttons, text …) which we can use to create the composable functions, that we can then reuse repeatedly, making UI creation a scalable process.

Composable functions are labelled with '@Composable' and do not return a value, typically emit a user interface blocks rendered by Compose Runtime.
Composable can be stateless or stateful:

- **Stateless**: driven only by the parameters they receive: if the function is invoked with the same parameters, it will produce the same result.
- **Stateful**: is when Composable creates, holds and modifies its own State, as mutable values declared with **remember** function in order to store them in memory.
  If the mutable value change this will cause recomposition.

To Create and manage the Ui the Jetpack Compose library work like this:

1. **Initial composition:** To create the Ui , the compose framework invoke the first composable function and record a set of composable functions that it invokes. In this way the **tree view** is built.
2. **Recomposition:** When a value passed as parameter or a state change, the composable function is executed again ,this change will affect only composable related to that value. This mechanism is **optimistic**: if a change occur while recomposition is in progress, the process is cancelled and recomposition restarted.

Composable functions are transformed by providing a composition context. They can be executed in any order , also in parallel. Typically they accept a **modifier** to manage decorations, when they accept lambda hierarchical composition is triggered, making the lambda a child of the caller.
Some foundational composable are provided by the library and as images and lazycolumn, but also some layout composable like buttons, col and row. Also **Scaffold** is useful because it implement material design visual structure, and topbar,bottombar ecc. Also jetpack navigation is fully supported.

[For example, when you need to create a counter and every time you push a button the counter will increase you need to use a stateful composable, in this way every time the counter variable is updated, the interface block will be updated displaying the new counter value]

2023-07-14

1.Dependency injection, how it's done in Hilt and the lifecycle of its components

Objects are often based on delegation patterns, part of their behavior is delegated to some other objects, named dependencies. This mechanism have several disadvantages:
It's not possible to compile one class if the dependency is unavailable and code can't be reused.

Dependency injection is a desing pattern that separates the concern of constructing objects from the one of using them: between the class and the dependencies the **Dependency injection framework** steps in. It is in fact useful in that situation where a class depends on multiple interfaces in order to decide who is responsible of providing the implementation.

Hilt is a DI framework which operates on top of Dagger (made by google). Using Hilt the programmer puts annotations in the code:

- **@HiltAndroidAPP** on the application and **@AndroidEntryPoint** in the MainActivity;
- **@Inject** near a constructor , or a decency destination;
- **@Singleton** near dependencies used all over the app; it is also possible to define the scope of the dependency and so the lifecycle:
    - @Singleton : survives until Application terminates;
    - @ActivityRetainedScoped objects survives the Activity Destroy;
    - @ActivityScoped survives until Activity destroy;
    - @FragmentScoped survives until Fragment detach;
    - @ViewModelScoped survives until ViewModel is onCleared();
    - @ViewScoped survives until View destruction;
    - @ServiceScoped survives until Service destroyed.

All these annotations is used by Hilt to specify the role played by a class and to mark the place where the dependency should be injected, at compile time Hilt use annotations to generate the code with @Component and @Subcomponent which Is then used by Dagger to inject the dependencies.

2.Different kinds of Animations and functions for Animations in Jetpack Compose.

Animation can be **functional**, if used to guide and inform the use, or **decorative** if used to support storytelling and emphasized the brand. Animation generally should follow the 12 principles of animation developed by Disney professionals.
In the perspective of how jetpack compose work there are:

- Animation that requires a **change in components** presents on the screen;
- Animation that requires **attributes of the element already present** , changing position, size, color.

Functions provided by jetpack compose are:

- **AnimatedVisibility**: show/hides content with fading,expanding etc;
- **Crossfade**: replaces one element with another with opacity transitions, use a state to manage the switch;
- **AnimatedContent**: Manage appearance and disappeareance, using based on target state;
- **Animatable**: encapsulate a value animated over time;
- **Animate*AsState**: is a composable function that animate values;
- **AnimationSpec**: is an inteface for animation-related parameters such as animated data type.

3.Describe the Information Architecture and related principal patterns

Information Architecture is a discipline that focuses on the organizations of information in digital products, like an architect applies to information space. This discipline manage to lay out each individual screen so users can easily find information they need. In addition is fundamental to create a flow that let users navigate through each screen .

The principal patterns are:

- **Hierarchy** : One index page, which links to other pages, that links subpages.
- **Hub and Spoke**: one index page (hub) with spokes. To go from one spoke to another you have to come back to index page. User can focus on one task at time.
- **Nested doll**: Index page has general overview of the content, pressing on it you move to the page with more details and so on.
- **Tabbed view**: Content is placed into different sections and users can switch between them using a toolbar.
- **Dashboard** : Index has different framework which shows the most important part of information.
- **Filtered view**: allow users to switch between alternatives view, filtering content.

These 6 principal patterns can be mixed to achieve the perfect user interface for your app.

2023-06-28

1. Describe the ViewModel lifecycle with respect to the activity/fragments it refers to

Like 2023-09-19 , Question 2: you can find it at page 4

2.What are Kotlin coroutines and how do they support cross thread invocations? Make a practical example

Like 2023-09-19 , Question 1: you can find it at page 3

3.Describe how a jetpack compose application deals with state. How is that integrated in a ViewModel architecture

Like 2023-09-19 , Question 3: you can find it at page 5

4.Write the 5 J.J Garret's plans of UX

Garret's five plans provide a conceptual framework for talking about user experience problem and the tools we use to solve it. Each plan is dependent on the plan below it . When the choices we make don't align with those above and below project derails:

1. **Strategy**: Make clear what we want to make out of this product and what user want from it;
2. **Scope**: translate what user needs the product to be into real **Requirements** for what content and functionality the product will offer to users;
3. **Structure**: Define how and when user go to that page ;
4. **Skeleton**: the skeleton of the page, so buttons, controls, photos, texts;
5. **Surface**: design that pleases the senses of user, while fulfilling all the goals of the outer four planes.

The right approach it's to finish the first plan before the next is finished.

2022-07-05

1.The main thread has the exclusive to perform some tasks, but it is limited to perform others. Explain how and why Kotlin coroutine are helpful in this specific case by providing a practical example.

Like 2023-09-19 , Question 1: you can find it at page 3

2.Explain what the role of the ViewModel is and how its life cycle is involved with the one of other android abstractions.

Like 2023-09-19 , Question 2: you can find it at page 4

3.Explain what is the Jetpack Compose library and what are its main abstractions.

Like 2023-09-19 , Question 3: you can find it at page 5

Possible Questions (invented by me)

1.Describe what are the Gestalt Principles and which are the most important in UI design

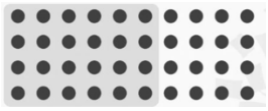
Gestalt Principles are principles of human perception that describe how humans group similar elements, recognize patterns and simplify complex images when we perceive object. The most important are:

- **Similarity** : the human eye tend to build a relationship between similar elements, in color, size and shape.
- **Continuity** : the human eye follows the path lines and curves of a design, and prefers to see a continuous flow of visual Elements rather than a separated object

- **Focal Point** : whatever stands out visually will capture and hold the viewer attention first (like a green square inside a set of blue circles)

- **Proximity** : things that are close together appear to be more related than things that are spaced far apart.

- **Common region**: Similar to proximity, when objects are in the same closed region, we perceive them as being grouped together.



- **Symmetry** : the human mind perceive objects as being symmetrical and forming around a center point. Like {} instead of {). Symmetry occurs naturally in our environment (like in butterfly or snails) and we also chose people based on these principles.
- **Figure-ground** : Refers to the tendency of the visual system to simplify a scene into the main object that we are looking at (the figure) and everything else that forms the background (or ground)

- **Closure** : When you see an image that has missing parts, your brain will fill in the blanks and make a complete image so you can still recognize the pattern